Grenoble INP - Ensimag, UGA

Introduction to Visual Computing

TP1



Gómez Rosales Laura Sofía
Lugo Nomesque José Francisco

Prof. Sergi Pujades
Prof. Julien Pansiot

September 20, 2025

Contents

1 Exercise 1 **2**

2 Exercise 2 **3**
2.1 Results 9

3 Exercise 3 **12**
3.1 Results 19

1 Exercise 1

Download the archive that contains the code of TP1.

1. Compile the programs using the Makefile and test *pgmtopgm* on man-drill.pgm by running

   ```
   % make
   % ./pgmtopgm images/mandrill.pgm results/mandril1.pgm
   ```

   What kind of conversion is performed ? Check the contents of man-drill.pgm and mandrill1.pgm with a text editor.

   A: It changes file format, P2(decimal) to P5(binary).

2. How is the image stored within the program ?

   A: It is stored in a structure call `pgm_file`, where it's define the array of gray scale values, besides its metadata.

3. What is the purpose of the functions pm_getint and pm_getrawbyte in the file Util.c ?

   A: These functions are used to read an image file as decimal as binary representation so they receive as Input a file and return char or int values.

   - `pm_getint:` A file is being read by the function `pgm_getc` which read return a char until the **EOF** then validates if there's any value that is not in range [0-9]. In case that all values are numbers just take the value casting to int datatype.
   - `pm_getrawbyte:` Read a file with `getc` function until **EOF** then cast and return unsigned char value.

4. What data types are involved when manipulating intensities ? in the case of ASCII decimals (P1, P2, P3) ? In the case of binary values (P4, P5, P6) ?

   - For ASCII's case, the data type used is `int`.
   - For Binary is `char` data type.

5. Which color is associated to the maximal value for each format?

   - **PBM:** It is black, with the max value equal 1
   - **PGM and PPM:** It is white, with the max vlue for 8-bit color depth equay to 255,and for 16-bit is 65536

6. Comment on the sizes of files in the two formats.

   - **P2** file size is **1.1 MB**
   - **P5** file size is **262.2 kB**
     This differences is because **P2** has 512x512 values in ASCII decimals, meanwhile **P5** is in binary having less lines, so it could be read directly by the machine.

2 Exercise 2

The PPM format allows to store images in RGB format. Check the file `ppm-torgb.c` defining the program `ppmtorgb`. Its goal is to convert a PPM image into three images with respect to the RGB channels.

1. First implement the read and write functions (`read_ppm` and `write_ppm`) in `Utils/Util.c`

   Note: In this section ChatGPT IA was used to generate ideas of how to access an struct based on the given code in `read_pgm` with this prompt 'Based on this function `read_ppm`code how would you access to the following struct typedef struct `ppm_file` '

   A: Based on the function `read_pgm`, the function `read_ppm` receive a file verifying that each part of the needed struct has the correct format : magic number (**P3** and **P6**), number of rows, number of cols, and max value. Then the **pixmap memory size** is calculated by the formula `cols * rows * sizeof(pixel)`, using `malloc` function.

   Reading is made going through two loops, and depending if value it's binary of ASCII decimals we call `pm_getrawbyte` or `pm_getint` functions to read the file components RGB and assign them to **pixmap struct.**

```
1
2  ppm_file read_ppm(char *filename) {
3      FILE *ifp;
4
5      int firstchar, rows, cols;
6      int is_raw;
7
8      int i,j;
9
10     ppm_file image;
11
12     /* Opening input file */
13     ifp = fopen(filename, "r");
14     if (ifp == NULL) {
15       printf("error in opening file %s\n", filename);
16       exit(1);
17     }
18
19     /*  Magic number reading */
20     firstchar = getc(ifp);
21     if (firstchar == EOF) {
22       pm_error("EOF / read error / magic number");
23     }
24
25     image.magic_number = getc(ifp);
```

```
26      if (image.magic_number == EOF) {
27        pm_error("EOF /read error / magic number");
28      }
29
30      if (image.magic_number != '3' && image.magic_number != '6') {
31        pm_error(" wrong file type ");
32      }
33
34      if (image.magic_number == '6'){
35        is_raw = 1;
36      } else{
37        is_raw = 0;
38      }
39
40      /* Reading image dimensions */
41      cols = pm_getint(ifp);
42      rows = pm_getint(ifp);
43      image.maxval = pm_getint(ifp);
44
45      image.rows = rows;
46      image.cols = cols;
47
48      /* Memory allocation  for each component of RGB */
49      image.pixmap = (pixel *) malloc(cols * rows * sizeof(pixel));
50
51      /* Reading binary or ASCII*/
52      for (i = 0; i < rows; i++) {
53        for (j = 0; j < cols; j++) {
54          if (is_raw) {
55            image.pixmap[i * cols + j].red = pm_getrawbyte(ifp);
56            image.pixmap[i * cols + j].green = pm_getrawbyte(ifp);
57            image.pixmap[i * cols + j].blue = pm_getrawbyte(ifp);
58          } else {
59            image.pixmap[i * cols + j].red = pm_getint(ifp);
60            image.pixmap[i * cols + j].green = pm_getint(ifp);
61            image.pixmap[i * cols + j].blue = pm_getint(ifp);
62          }
63        }
64      }
65
66      /* Closing input file */
67      fclose(ifp);
68
69      return image;
70  }
71
```

A: `write_ppm` function is based on the `write_pgm` function taking an image to write it in one file as binary as ASCII decimal, besides RGB components for PPM have a pixmap struct that should be taken into account when loops are iterated, each component should be written in their respective file.

```
1
2  void write_ppm(ppm_file image, char *filename) {
3
4    FILE *ofp;
5    int is_raw, i, j;
6
7    /* Opening output file */
8    ofp = fopen(filename, "w");
9    if (ofp == NULL) {
10     printf("error in opening file %s\n", filename);
11     pm_error(" FILE OPEN ERROR");
12   }
13
14   if (image.magic_number == '3') {
15     is_raw = 0;
16   } else {
17     is_raw = 1;
18   }
19
20   if (is_raw) {
21     fprintf(ofp, "P6\n");
22   } else {
23     fprintf(ofp, "P3\n");
24   }
25
26   fprintf(ofp, "%d %d \n", image.cols, image.rows);
27   fprintf(ofp, "%d\n", image.maxval);
28
29   /* Write each RGB component in filename file */
30   for (i = 0; i < image.rows; i++)
31     for (j = 0; j < image.cols; j++)
32       if (is_raw){
33           fprintf(ofp, "%c", image.pixmap[i * image.cols + j].red);
34           fprintf(ofp, "%c", image.pixmap[i * image.cols + j].green);
35           fprintf(ofp, "%c", image.pixmap[i * image.cols + j].blue);
36         }
37       else{
38         fprintf(ofp, "%d ", image.pixmap[i * image.cols + j].red);
39         fprintf(ofp, "%d ", image.pixmap[i * image.cols + j].green);
40         fprintf(ofp, "%d ", image.pixmap[i * image.cols + j].blue);
41       }
42
43   /* Closing output file */
44   fclose(ofp);
45
46 }
```

2. Then each channel should be stored in a PPM image with zeros on the other channels. Implement the body of the function *split_channels* in TP1 TP1_funcs.c to complete the program. Test it by running

   ```
   % ./ppmtorgb images/mandrill.ppm results/mandrillR.ppm
   results/mandrillG.ppm results/mandrillB.ppm
   ```

A: `split_channels` function take as parameters one image stored in `ppm_file` struct and one image by each component RGB, then take shared attributes and assign them to each image header, finally for pixmap.

```
1
2  void split_channels(ppm_file image,
3                      ppm_file *image_r, ppm_file *image_g, ppm_file *
       image_b) {
4
5      // Define ppm_file for red channel
6      image_r -> rows = image.rows;
7      image_r -> cols = image.cols;
8      image_r -> magic_number = image.magic_number;
9      image_r -> maxval = image.maxval;
10
11     // Define ppm_file for green channel
12     image_g -> rows = image.rows;
13     image_g -> cols = image.cols;
14     image_g -> magic_number = image.magic_number;
15     image_g -> maxval = image.maxval;
16
17     // Define ppm_file for blue channel
18     image_b -> rows = image.rows;
19     image_b -> cols = image.cols;
20     image_b -> magic_number = image.magic_number;
21     image_b -> maxval = image.maxval;
22
23     // Allocates memory for each component
24     image_r -> pixmap = (pixel *) malloc(image.cols * image.rows *
       sizeof(pixel));
25     image_g -> pixmap = (pixel *) malloc(image.cols * image.rows *
       sizeof(pixel));
26     image_b -> pixmap = (pixel *) malloc(image.cols * image.rows *
       sizeof(pixel));
27
28     for (size_t i = 0; i < image.rows; i++){
29         for (size_t j = 0; j < image.cols; j++){
30             // Red component image after split
31             image_r -> pixmap[i * image.cols + j].red
32                 = image.pixmap[i * image.cols + j].red;
33             image_r -> pixmap[i * image.cols + j].green = 0;
34             image_r -> pixmap[i * image.cols + j].blue = 0;
```

```
35
36              // Green component image after split
37              image_g -> pixmap[i * image.cols + j].red = 0;
38              image_g -> pixmap[i * image.cols + j].green
39                  = image.pixmap[i * image.cols + j].green;
40              image_g -> pixmap[i * image.cols + j].blue = 0;
41
42              // Blue component image after split
43              image_b -> pixmap[i * image.cols + j].red = 0;
44              image_b -> pixmap[i * image.cols + j].green = 0;
45              image_b -> pixmap[i * image.cols + j].blue
46                  = image.pixmap[i * image.cols + j].blue;
47          }
48      }
49  }
50
```

3. Check the file ppmtopgm.c defining the program ppmtopgm. Its goal is to convert a PPM image into a grayscale image. Implement the body of the function *rgb_to_gray* in *TP1 TP1_funcs.c* . Test it by running

   A: The function `rgb_to_gray` takes 3 RGB images (one by each channel) and extract shared attributes such as rows, cols, magic number and maxVal then applying formula to mix the channels it's possible to get each value in grayscale then they are stored in a `ppgm_file` file.

```
% ./ppmtopgm images/mandrill.ppm results/mandrill_gray.pgm
```

```
1
2  void rgb_to_gray(ppm_file image, pgm_file *image_gray, float
       brightness){
3
4      // Define values for gray scale image
5      image_gray -> rows = image.rows;
6      image_gray -> cols = image.cols;
7      image_gray -> magic_number = image.magic_number;
8      image_gray -> maxval = image.maxval;
9
10     int conversion = 0;
11     image_gray -> graymap = (gray *) malloc(image.cols * image.rows *
       sizeof(gray));
12
13     for (size_t i = 0; i < image.rows; i++){
14         for (size_t j = 0; j < image.cols; j++){
15             conversion = 0.299 * image.pixmap[i * image.cols + j].red
       + 0.587 * image.pixmap[i * image.cols + j].green + 0.114 * image.
       pixmap[i * image.cols + j].blue;
```

```
16              image_gray -> graymap[i * image.cols + j] = (float)
    conversion * brightness;
17          }
18      }
19  }
20
```

4. Use the variable "brightness" in the conversion process to obtain a brighter or darker image. Test it by running

A: The function `rgb_to_gray` has an optional parameter in input which taking the formula used in the last exercise it's possible to modify to obtain a brighter or darker image.

```
% ./ppmtopgm images/mandrill.ppm results/mandrill_gray_bright.pgm 1.2

% ./ppmtopgm images/mandrill.ppm results/mandrill_gray_dark.pgm 0.8
```

2.1 Results

- **Exercise 2 PGM to RGB images:**
  ```
  % ./ppmtorgb images/mandrill.ppm results/mandrillR.ppm
    results/mandrillG.ppm results/mandrillB.ppm
  ```

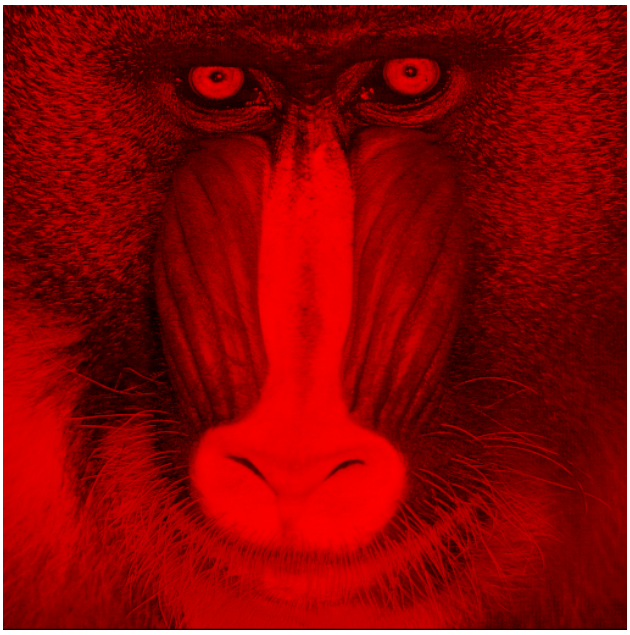


Figure 1: Red channel



Figure 2: Green channel

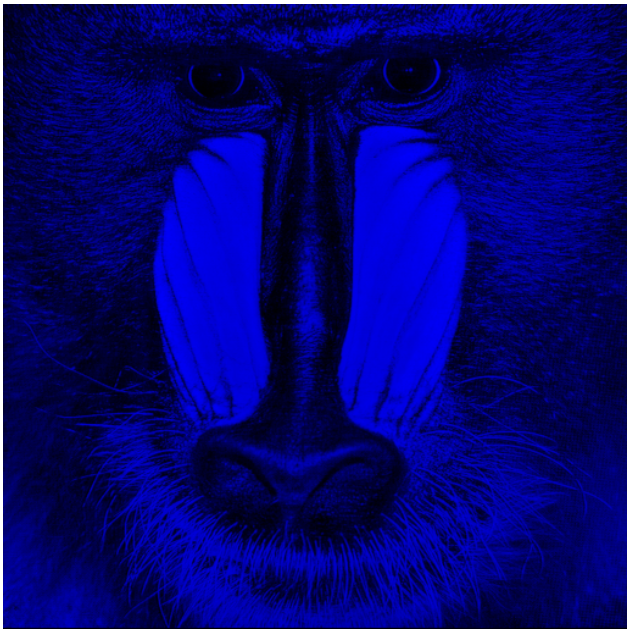


Figure 3: Blue channel

- **Exercise 2 PPM to Grayscale :**
  `% ./ppmtopgm images/mandrill.ppm results/mandrill_gray.pgm`

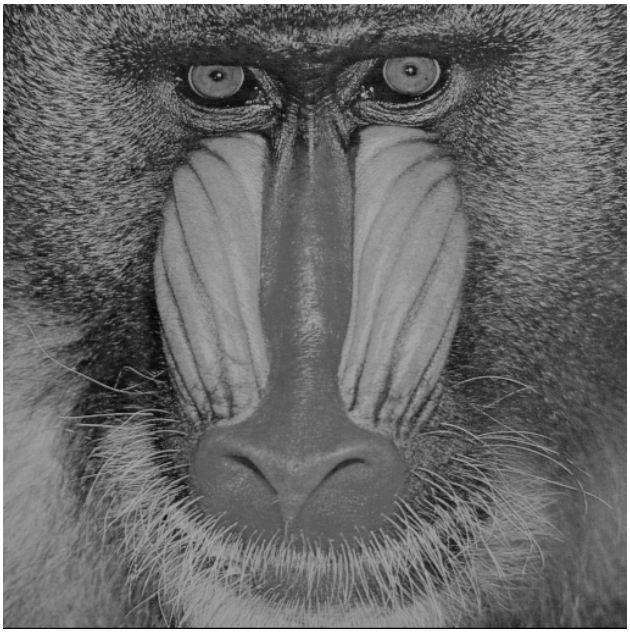


Figure 4: Grayscale image

Note: To see the results files, you can check it in the folder `results`

3 Exercise 3

An extension of the netpbm formats to transparent images exist, i.e. images for which there is an additional alpha channel encoding transparency. It is called PAM (extension .pam for files) for Portable Arbitrary Map.

Note: In this section, we use AI Studio to clarify how PAM files works, also understand better the objective of each Bonus creating a logical resolution track. The prompts were asking the IA to confirm if our logical track was correct. Some minor errors were corrected by Github Copilot.

1. Look over the web and give the main principles of this extension, e.g. file header and pixel format.

   A: Source: https://netpbm.sourceforge.net/doc/pam.html
   Header

   Magic Number: P7

   Width: Number of columns

   Heigh: Number of rows

   Depth: Number of channels

   Maxval: Max value of theimage

   Tupltype: Gives a name to the structure of the pixel data, defining what the numbers in the image means (e.g. RGB, GRAYSCALE, RGB_ALPHA)

   Endhdr: This is the last line in the header

   Notes:

   - **suffix:** `.pam`
   - PAM is a "lowest common denominator" format. It isn't just for images; it's designed to store any kind of data that can be arranged in a two-dimensional grid.
   - Height, width, depth, and maxval are at least 1.
   - Height, width, and depth have no defined maximum, but processors and generators of images usually have their own limitations.
   - The maxval of an image is never greater than 65535.

2. **Bonus 1:** write a program `ppmtopam` that converts a PPM image into a PAM image with the alpha value (transparency) set to 255 for all pixels.

```
% ./ppmtopam images/mandrill.ppm results/mandrill.pam

% ffmpeg -i results/mandrill.pam results/mandrill.png
```

3. **Bonus 2:** improve the ppmtopam program so that it also takes a PGM image and creates a PAM image with the colors of the PPM image and the alpha value (transparency) from the PGM image.

```
% ./ppmtopam images/mandrill.ppm images/mandrill.pgm
   results/mandrill_masked.pam

% ffmpeg -i results/mandrill_masked.pam results/mandrill_masked.png
```

A: To resolve Bonus 1 and 2, a PAM file handler was developed. This involved defining a new data structure and implementing functions for reading and writing PAM files, which could then be used by a converter program.

The first step was to define a data structure capable of representing a PAM image in memory. This was added to the `Utils/type.h` file. The `pam_file` struct contains fields for the header metadata, and a pointer, `map`, which corresponds to the raster.

```
1
2   // A PAM file, containing an arbitrary map and its metadata
3   typedef struct {
4     char *map;
5     char magic_number;
6     int rows;
7     int cols;
8     int depth;
9     int maxval;
10    char *tuple_type;
11  } pam_file;
12
```

Then, in `Utils/Utils.c` is implemented the read and write functions based on `ppm's` ones, adding the new structure components: depth, maxval and tupletype.

Also, `read_pam` function is responsible for parsing the PAM header keywords. And allocating an amount of memory for the raster by calculated with the formula: `cols * rows * depth`. Since the PAM raster is always binary, the function reads the raw bytes directly by `pm_getrawbyte`

```
1
2  pam_file read_pam(char *filename) {
3      pam_file image;
4      FILE *ifp;
5      int i, j;
6
7      /* Opening input file */
8      ifp = fopen(filename, "r");
9      if (ifp == NULL) {
10         printf("error in opening file %s\n", filename);
11         pm_error("FILE OPEN ERROR");
12     }
13
14     /* Magic number reading */
15     image.magic_number = getc(ifp);
16     if (image.magic_number == EOF) {
17         pm_error("EOF / read error / magic number");
18     }
19
20     if (image.magic_number != '7') {
21         pm_error("wrong file type");
22     }
23
24     /* Reading image dimensions */
25     image.cols = pm_getint(ifp);
26     image.rows = pm_getint(ifp);
27     image.depth = pm_getint(ifp);
28     image.maxval = pm_getint(ifp);
29
30     /* Memory allocation */
31     image.map = (char *) malloc(image.cols * image.rows * image.depth
       * sizeof(char));
32     if (!image.map) {
33         pm_error("Memory allocation failed");
34     }
35
36     /* Reading */
37     for (i = 0; i < image.rows; i++) {
38         for (j = 0; j < image.cols; j++) {
39             int k;
40             for (k = 0; k < image.depth; k++) {
41                 image.map[(i * image.cols + j) * image.depth + k]
42                     = pm_getrawbyte(ifp);
43             }
44         }
45     }
46
47     /* Closing input file */
48     fclose(ifp);
49     return image;
50 }
51
```

For write function is important to print in file in order and the component name for each one, also end the header with `ENDHDR`. Then save the map information according to (i, j, k) elements position.

```
1
2  void write_pam(pam_file image, char *filename) {
3      FILE *ofp;
4      int i, j, k;
5
6      /* Opening output file */
7      ofp = fopen(filename, "w");
8      if (ofp == NULL) {
9          printf("error in opening file %s\n", filename);
10         pm_error("FILE OPEN ERROR");
11     }
12
13     fprintf(ofp, "P7\n");
14     fprintf(ofp, "WIDTH %d\n", image.cols);
15     fprintf(ofp, "HEIGHT %d\n", image.rows);
16     fprintf(ofp, "DEPTH %d\n", image.depth);
17     fprintf(ofp, "MAXVAL %d\n", image.maxval);
18     fprintf(ofp, "TUPLETYPE %s\n", image.tuple_type);
19     fprintf(ofp, "ENDHDR\n");
20
21     for (i = 0; i < image.rows; i++) {
22         for (j = 0; j < image.cols; j++) {
23             for (k = 0; k < image.depth; k++) {
24                 fputc(image.map[(i * image.cols + j) * image.depth +
25                     k], ofp);
26             }
27         }
28     }
29
30     /* Closing output file */
31     fclose(ofp);
32 }
33
34
```

In the main project directory, it is created the script `ppmtopam.c`. Here, the `main` function acts as a controller, handling arguments and orchestrating the file I/O and conversion steps.
The logic process is as follows:

1. Parse command-line arguments to determine the input and output file paths
2. Determine the operational mode based on the number of arguments provided. Three arguments is for the full 255 alpha conversion (Bonus 1), while four arguments indicates that a PGM mask should be used for the alpha channel (Bonus 2).
3. Read the input image file(s) into memory
4. Call a dedicated conversion function to generate the final PAM data structure
5. Write the resulting PAM structure in an output file `.pam`

```
1
2  int main(int argc, char* argv[]) {
3      // Image definitions
4      ppm_file image;
5      pgm_file image_gray;
6      pam_file image_pam;
7
8      /* Arguments */
9      if (argc == 3){
10         /* Opening input file */
11         image = read_ppm(argv[1]);
12
13         // Conversion
14         ppm_to_pam(image, &image_pam, NULL);
15
16         /* Write output file */
17         write_pam(image_pam, argv[2]);
18
19     } else if (argc == 4){
20
21         image = read_ppm(argv[1]);
22         image_gray = read_pgm(argv[2]);
23
24         ppm_to_pam(image, &image_pam, &image_gray);
25
26         write_pam(image_pam, argv[3]);
27
28     } else {
29         printf("\nUsage : %s file_in file_out \n\n", argv[0]);
30         exit(0);
31     }
32
33     return 0;
34 }
35
```

The conversion is handled by the `ppm_to_pam` function, implemented in `TP1/TP1_funcs.c`. This function is designed to handle both bonus cases by treating the **PGM mask** as an optional parameter, identified by a `NULL` pointer.

The function first write the metadata for the output `pam_file` struct. The depth is set to 4 to account for the new alpha channel, and the `tuple_type` is set to `"RGB_ALPHA"`.

In 2 loops, for each R[0] G[1] B[2] position is assign its value obtained from the pixmap. Then the Alpha[3] channel is added according to:

- If `image_gray` was giving, the gray mask is going to be created assigning the graymap value in that position to the Alpha channel
- Else, the Alpha channel is going to be the max value for PAM, 255

```
1
2  void ppm_to_pam(ppm_file image, pam_file *image_pam, pgm_file *
       image_gray) {
3      int i, j;
4
5      // Metadata
6      image_pam->magic_number = '7';
7      image_pam->rows = image.rows;
8      image_pam->cols = image.cols;
9      image_pam->depth = 4;
10     image_pam->maxval = image.maxval;
11     image_pam->tuple_type = "RGB_ALPHA";
12
13     // Memory allocation
14     image_pam->map =
15         (char *) malloc(image.cols * image.rows * image_pam->depth
16         * sizeof(char));
17     if (!image_pam->map) {
18         pm_error("Memory allocation failed");
19     }
20
21     // Copy data
22     for (i = 0; i < image.rows; i++) {
23         for (j = 0; j < image.cols; j++) {
24             // Red
25             image_pam->
26                 map[(i * image.cols + j) * image_pam->depth + 0] =
27                 image.pixmap[i * image.cols + j].red;
28
29             // Blue
30             image_pam->
31                 map[(i * image.cols + j) * image_pam->depth + 1] =
32                 image.pixmap[i * image.cols + j].green;
33
34             // Green
35             image_pam->
```

```
                map[(i * image.cols + j) * image_pam->depth + 2] =
                image.pixmap[i * image.cols + j].blue;

            // Alpha
            if (image_gray != NULL)
                // PGM Mask
                image_pam->
                    map[(i * image.cols + j) * image_pam->depth + 3]
                    = image_gray->graymap[i * image.cols + j];
            else {
                image_pam->
                    map[(i * image.cols + j) * image_pam->depth + 3]
                    = 255;
            }
        }
    }
}

```

3.1 Results

Note: To see the results files, you can check it in the folder `results`

- **General Header:**
  P7
  WIDTH 512
  HEIGHT 512
  DEPTH 4
  MAXVAL 255
  TUPLETYPE RGB_ALPHA
  ENDHDR

- **Bonus 1:**
  ```
  % ./ppmtopam images/mandrill.ppm results/mandrill.pam

  % ffmpeg -i results/mandrill.pam results/mandrill.png
  ```



Figure 5: PAM image - ALPHA 255

- **Bonus 2:**

```
% ./ppmtopam images/mandrill.ppm images/mandrill.pgm
   results/mandrill_masked.pam

% ffmpeg -i results/mandrill_masked.pam results/mandrill_masked.png
```



Figure 6: PAM image - ALPHA 255 graymap